

Wireless Communications

Wi-Fi Jammer Implementation on ath9k



Nikas Ioannis Iason
ioannikas@uth.gr
3771

Mpalamotis Panagiotis
pabalamotis@uth.gr
3726

Tsiantos Dimitrios
dtsiantos@uth.gr
3796

June 2026

Contents

1	Introduction	3
2	Motivation	3
3	Design and Implementation	3
3.1	Driver Modification Overview	3
3.2	Module Parameters	3
3.3	selfish_mode: EDCA Manipulation	4
3.4	disable_backoff: Backoff Bypass	4
3.5	chanel_idle: Forced Channel Idle	4
3.6	selfish_txop_us: TXOP Override	5
3.7	Channel Switch Implementation	5
4	Experimental Methodology	6
4.1	Testbed and Common Configuration	6
4.2	Topologies	6
4.3	Driver Configurations	7
4.4	Measurement and Post-processing	8
5	Results and Evaluation	8
5.1	Experiments 1 and 2: Single-node Reference Throughput	8
5.2	Experiment 3: Single AP with Fair and Unfair Stations	8
5.3	Experiment 4: Two APs with Fair and Unfair Stations	10
5.4	Experiment 5: Channel-switch Follow with Backoff Bypass	13
5.5	Packet-loss Interpretation	14
6	Conclusion	15

1 Introduction

Wireless local area networks based on IEEE 802.11 are designed around a shared-medium access model, where multiple stations contend for access to the same radio channel. In normal operation, fairness between stations is achieved through carrier sensing, randomized backoff, and the Enhanced Distributed Channel Access (EDCA) mechanism in general. These mechanisms reduce collisions and allow multiple users to coexist, but they also rely on the assumption that all stations follow the protocol rules.

This project investigates what happens when this assumption isn't true at the driver level. Specifically, we modify the ath9k wireless driver in order to create configurable selfish behavior. This directly changes how the wireless device participates in channel contention and allows us to study how a station can gain an unfair transmission advantage by manipulating parameters such as contention window values, backoff behavior, channel-idle sensing, and transmission opportunity duration.

The goal of the work is to explore the impact of this MAC-layer selfishness in a controlled testbed. By comparing baseline behavior against several increasingly aggressive driver modifications, we evaluate how fairness can be affected by a single misbehaving node and how different selfish mechanisms influence throughput, contention, and coexistence with other stations.

2 Motivation

The motivation for this project comes from the observation that 802.11 fairness is not enforced only by the access point or by higher network layers. It also depends on low-level MAC behavior inside the wireless device. Vanhoef and Piessens showed in *Advanced Wi-Fi Attacks Using Commodity Hardware* that commodity Atheros hardware with modifiable firmware can be used to implement low-layer Wi-Fi attacks, including attacks that change how a device accesses or disrupts the shared channel [1]. Their work demonstrated that practical attacks do not necessarily require specialized radio equipment; instead, changing driver or firmware behavior can be enough to violate the assumptions of cooperative channel access.

Our work takes inspiration from that idea, but focuses on a controlled experimental study of selfish channel access rather than on building a general attack framework. In particular, we use the ath9k driver as the modification point and expose several parameters that let the same station behave either normally or unfairly.

3 Design and Implementation

3.1 Driver Modification Overview

The implementation is based on modifying the Linux ath9k driver used by Atheros wireless cards. The changes are applied at the driver level, so the experimental station can alter its MAC behavior without changing the applications that generate traffic. This is useful for the project because it provides much more control over the fairness of the channel use and also the same traffic generator and topology can be used for both fair and unfair cases, while only the driver parameters change.

The main modifications are placed in the ath9k_hw path. The EDCA queue parameters and TXOP duration are changed in `ath9k_hw_resettxqueue()`, because this function writes the queue contention values to the hardware registers. The backoff-bypass and forced-idle options are applied in `ath9k_hw_init_global_settings()`, so that they are re-applied whenever the hardware is initialized. We also modified the TX DMA abort path so that the driver does not accidentally clear the experimental register bits when the corresponding module parameters are enabled.

This design keeps the unfair behavior configurable. A station can be loaded with all parameters disabled and behave as a normal station, or it can be loaded with one or more selfish options enabled. This makes the implementation suitable for controlled experiments, because each mechanism can be tested separately and then compared against the baseline.

3.2 Module Parameters

The modified driver exposes the selfish behavior through module parameters. The parameters are passed when loading `ath9k_hw`, and their values are also visible through the Linux module parameter interface. The parameters used in the experiments are:

- `selfish_mode`: enables EDCA parameter manipulation by reducing the contention window and AIFS values to their minimum.
- `disable_backoff`: enables the hardware bit that makes the device ignore the normal backoff procedure.
- `chanel_idle`: enables the hardware bit that forces the hardware to treat the channel as idle.
- `selfish_txop_us`: sets a custom TXOP/burst duration in microseconds. A value of zero disables the override.

Using module parameters avoids recompiling the driver for every experiment. For example, a fair station can be loaded with all options set to zero, while an unfair station can be loaded with only one specific mechanism enabled. This separation is important because it allows us to measure the contribution of each modification independently.

3.3 `selfish_mode`: EDCA Manipulation

The `selfish_mode` option changes the queue EDCA contention parameters before they are written to the hardware. In normal 802.11 operation, each station waits for an AIFS interval and then selects a random backoff value from the contention window. Larger contention windows reduce collision probability but also make the station wait longer before transmitting.

When `selfish_mode` is enabled, the driver sets the minimum contention window to zero, the maximum contention window to zero, and the AIFS value to one. This makes the station much more aggressive, because after the medium becomes available it does not perform the same random waiting process as a normal station. The modified values are written through the normal queue reset path, so the hardware uses them as its active contention parameters.

This mechanism represents the least invasive selfish behavior in the project. It does not completely disable carrier sensing, but it gives the modified station a strong advantage whenever multiple stations are competing for access after the channel becomes idle.

3.4 `disable_backoff`: Backoff Bypass

The `disable_backoff` option is more aggressive than simply reducing the contention window. It sets the `AR_D_GBL_IFS_MISC_IGNORE_BACKOFF` hardware bit, which causes the device to ignore the normal backoff behavior. Instead of only making the selected backoff value smaller, this option targets the backoff mechanism itself.

The bit is set during hardware global initialization. This location was chosen so that the option is restored after driver or hardware reinitialization. Since the original driver may clear this bit during TX DMA abort handling, the abort path was also changed to clear the bit only when `disable_backoff` is not enabled. This prevents the experiment from being silently disabled by later driver activity.

Experimentally, this parameter is useful for separating EDCA parameter manipulation from a direct backoff bypass. If `selfish_mode` improves the station's access probability by reducing the waiting range, `disable_backoff` tests what happens when the station bypasses a larger part of the contention process.

3.5 `chanel_idle`: Forced Channel Idle

The `chanel_idle` parameter forces the hardware to consider the wireless medium idle by setting the `AR_DIAG_FORCE_CH_IDLE_HIGH` bit. This affects the carrier sensing part of 802.11 channel access. A normal station should defer its transmission when it senses that the medium is busy. With this option enabled, the modified station can behave as if the channel is free even when other transmissions may be present.

This is one of the most aggressive modifications because it interferes with the basic behavior of sensing before transmitting of 802.11. While it can increase the unfair station's willingness to transmit, it can also increase collisions and reduce total network efficiency. For this reason, the option is tested separately from the simpler EDCA and backoff modifications.

As with `disable_backoff`, the bit is applied during hardware initialization and protected from being cleared unintentionally in the TX DMA abort path. This ensures that the value loaded through the module parameter remains active during the experiment.

3.6 selfish_txop_us: TXOP Override

The `selfish_txop_us` parameter controls how long the station may occupy the channel after it wins access. Unlike the contention-window and backoff modifications, TXOP manipulation does not necessarily help the station win the medium more often. Instead, it allows the station to keep the medium for a longer burst once it has gained access.

The implementation changes the queue burst time before the driver writes the `AR_DCHNTIME` register. The value is provided in microseconds and is capped by the hardware duration mask, so invalidly large values are not written directly to the register. A value of 0 disables the override and leaves the normal driver behavior unchanged.

This parameter allows us to test a different type of selfishness. The previous options make the station more aggressive before transmission, while `selfish_txop_us` changes the behavior after channel access has already been obtained. In the experiments, this helps distinguish between winning the channel more frequently and holding the channel for longer periods.

3.7 Channel Switch Implementation

In addition to the selfish MAC parameters, the experiment framework also supports channel-switching scenarios. The purpose of this mechanism is to study whether an unfair node or neighboring unfair network can continue affecting a fair link even when the fair access point changes channel. This is useful because channel changes are a realistic way for an AP to avoid interference, but they are only effective if the interfering node does not follow the move.

The channel switch is initiated by the experiment runner on AP1. After a configured delay, the script executes a `hostapd_cli chan_switch` command on the AP. In our experiments, AP1 starts on channel 1 and then switches to channel 6 after 30 s. The command makes `hostapd` announce the upcoming move using Channel Switch Announcement (CSA) beacons before changing the operating frequency. The script records the time of the command, the target channel, the AP status before and after the switch, and relevant driver or kernel messages. These logs are later used during post-processing to mark the channel-switch event on the result plots.

For the two-AP scenario, the implementation also includes a following mechanism for AP2. A monitor interface on the unfair side listens for AP1 beacons and parses CSA information from them. When it detects that AP1 is moving to a new channel, it notifies a small listener running on AP2. The listener then issues the corresponding `hostapd_cli chan_switch` command on AP2, so that the unfair AP/STA link can move to the same channel as the fair AP. This preserves the idea of two independent BSSs (Basic Service Sets) while still allowing the unfair link to continue sharing the same RF medium after AP1 changes channel.

A separate single-node CSA-following script was also implemented for jammer-style experiments. In that mode, one monitor-mode interface is used both to sniff CSA beacons and to transmit injected 802.11 frames. When a CSA beacon is detected, the script estimates the switch time from the CSA countdown and beacon interval, stops transmission, retunes the monitor interface to the new channel, and resumes transmission. This mode is useful for demonstrating a local channel-following transmitter, but it is different from the AP2 case because it does not represent normal associated data traffic or produce meaningful `iperf` throughput.

wlan.fc.type_subtype == 0x08 && wlan.bssid == 04:54:53:00:dd:b2 && wlan.tag.number == 37					
No.	Time	Source	Destination	Protocol	Length Info
76887	12.039686	Apple_00:dd:b2	Broadcast	802.11	229 Beacon frame,
77756	12.140621	Apple_00:dd:b2	Broadcast	802.11	229 Beacon frame,
78626	12.243868	Apple_00:dd:b2	Broadcast	802.11	229 Beacon frame,
79507	12.348491	Apple_00:dd:b2	Broadcast	802.11	229 Beacon frame,
Tag length: 4					
DTIM count: 1					
DTIM period: 2					
Bitmap control: 0x00					
Partial Virtual Bitmap: 00					
Tag: Channel Switch Announcement Mode: 0, Number: 6, Count: 4					
Tag Number: Channel Switch Announcement (37)					

Figure 1: Captured CSA beacon frame from an AP

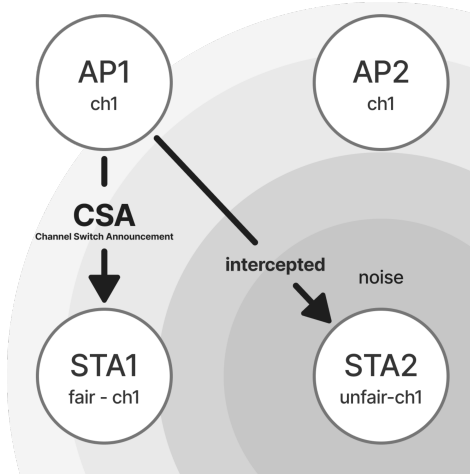


Figure 2: Interception of a CSA beacon frame

4 Experimental Methodology

The experiments were performed on the NITLab Nitros wireless testbed using Linux nodes equipped with Atheros wireless cards and the modified `ath9k/ath9k_hw` driver [2]. The same experiment runner, `run.sh`, was used for node preparation, driver loading, AP/STA configuration, traffic generation, log collection, and post-processing. This was done to keep the procedure reproducible and to ensure that each run stored the relevant configuration together with its measurement logs.

4.1 Testbed and Common Configuration

All reported traffic experiments used UDP `iperf` with an offered bitrate of 150 Mbit/s per active station and a duration of 60 s. The AP-side `iperf` servers listened on separate ports for the two stations: port 5004 for the fair station and port 5003 for the unfair station. Throughput, jitter, and packet loss were taken from the receiver/server-side `iperf` logs, because in UDP experiments the receiver summary is the authoritative evidence of how many packets actually arrived. The client-side logs were used mainly to recover start timestamps and align the time axis of concurrent flows.

The wireless network was configured in IEEE 802.11n mode on SSID `tsiantos`. In the collected runs the initial RF channel was channel 1, while the channel-switch experiments moved the AP1/fair link to channel 6 after 30 s. The default IP assignment was 192.168.2.1 for AP1 and 192.168.2.2 for the fair station. In the two-AP topology, AP2 used the separate subnet 192.168.3.0/24, with AP2 at 192.168.3.1 and the unfair station at 192.168.3.2. Keeping the two AP links on separate IP subnets avoids ARP and routing ambiguity while still allowing the two WLANs to share the same RF channel and interfere at the MAC/PHY layer.

Before each measurement, the runner loaded the selected driver parameters, configured the APs with `hostapd`, connected the stations to their assigned SSIDs, and saved setup logs under each experiment directory. These setup logs include the observed link information from `iw`, such as signal strength and transmit bitrate.

4.2 Topologies

Four experimental topologies were used:

1. **Single AP, single fair station.** AP1 served only the fair station. This topology provides a baseline sanity check for the unmodified driver and for the measurement pipeline. Since there is no competing unfair transmitter, it establishes the expected behavior of a normal AP–STA link under the same offered 150 Mbit/s UDP load.
2. **Single AP, single unfair station.** AP1 served only the unfair station, which was loaded with the selected selfish-driver parameters. This topology isolates the behavior of the modified station without competition from a fair station. It is useful for checking whether the unfair configuration can transmit correctly, what throughput it achieves by itself, and whether the driver modifications introduce loss or instability even when the medium is not shared with another active STA.

3. **Single AP, two stations.** Both the fair and unfair stations associated with AP1 and transmitted UDP traffic toward AP1 concurrently. The fair station used the normal driver parameters, while the unfair station used the selected selfish-driver configuration. The fair station was started first and the unfair station was started 10 s later. This staggered start makes it possible to observe the fair link before and after the unfair station joins the contention, and therefore directly shows the effect of selfish MAC behavior inside a single BSS.
4. **Two APs, two stations.** The fair station associated with AP1 and the unfair station associated with AP2. Both APs operated on the same initial RF channel so that the two independent links still contended for the wireless medium, but each link had its own SSID and IP subnet. This topology was used to study coexistence between two independent BSSs. In the channel-switching runs, AP1 performed a channel switch from channel 1 to channel 6 at 30 s. An AP2 follow mechanism could then switch AP2 after the unfair station detected the AP1 beacon/channel-change event, allowing the unfair link to continue following the fair link's channel.

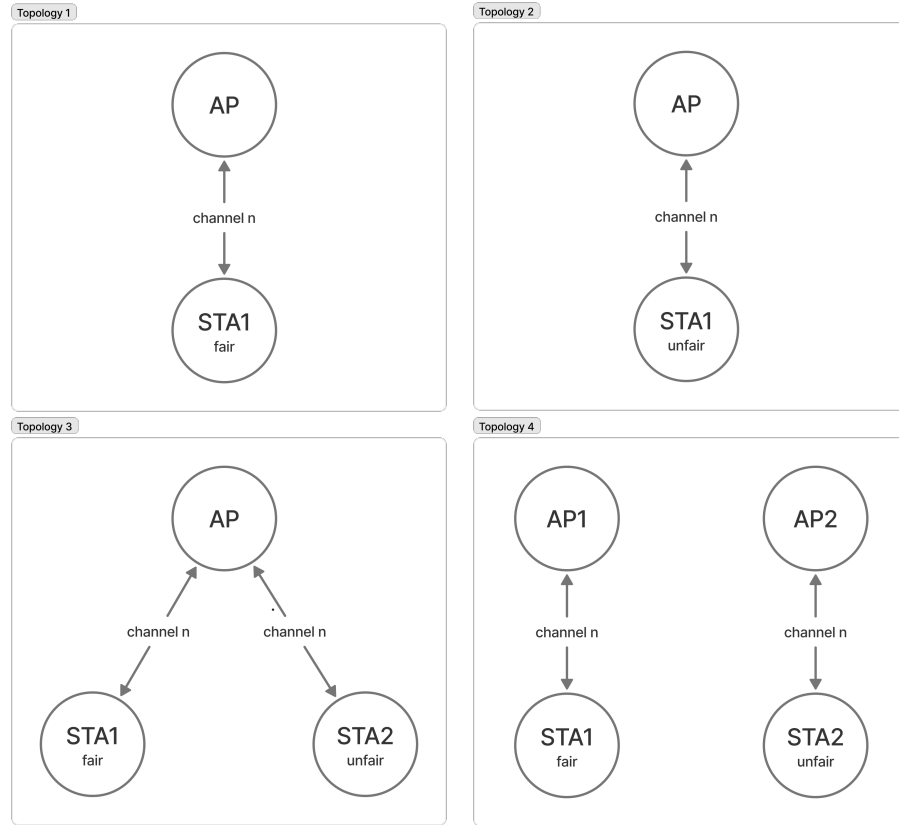


Figure 3: The four topologies used for the experiments.

4.3 Driver Configurations

The fair station was always loaded with the baseline parameters:

```
selfish_mode=0 disable_backoff=0 chanel_idle=0 selfish_txop_us=0
```

The unfair station was configured differently depending on the run. The tested configurations included a baseline/off case and more aggressive combinations of the implemented mechanisms. In particular, the collected experiments include runs with backoff bypass only, backoff bypass combined with forced channel-idle sensing, and the strongest configuration combining EDCA manipulation, backoff bypass, and a custom TXOP of 8100 μ s. The most aggressive configuration is:

```
selfish_mode=1 disable_backoff=1 chanel_idle=1 selfish_txop_us=8100
```

4.4 Measurement and Post-processing

Each experiment directory contains a configuration snapshot, AP and station setup logs, the raw `iperf` logs, and the generated summaries. The parser extracts the per-second receiver bandwidth and UDP packet-loss fields from the AP-side `iperf` server logs.

5 Results and Evaluation

For all numeric comparisons, the values are taken from the final receiver-side UDP `iperf` summaries. The offered traffic was 150 Mbit/s; therefore, values below that rate represent the effective throughput that the AP actually received over the wireless link.

5.1 Experiments 1 and 2: Single-node Reference Throughput

The first two experiments were used only as reference measurements. They do not evaluate fairness, because only one station is active and there is no competing transmitter. Their purpose is to estimate the maximum practical throughput of each node under ideal conditions and to verify that both the fair and unfair stations can transmit correctly before the contention experiments.

Table 1: Single-node reference throughput under ideal conditions. The reported value is the average of the maximum receiver-side throughput observed in the single-node runs.

Experiment	Active station	Avg. receiver bandwidth (Mbit/s)
1	Fair station only	87.7
2	Unfair station only	96.5

The unfair station has a slightly higher single-node average, but the important observation is that both stations are able to reach approximately the same order of throughput when they do not compete with another transmitter. Therefore, the large differences observed in the next experiments are not explained simply by one node being unable to transmit. They are mainly caused by the contention behavior introduced when the fair and unfair stations are active at the same time.

5.2 Experiment 3: Single AP with Fair and Unfair Stations

In the third experiment, both stations were associated with the same AP. The fair station started first and the unfair station started 10 s later. Table 2 summarizes the final receiver-side measurements for each unfair-driver configuration.

Table 2: Experiment 3 results: single AP, two stations. Values are computed from receiver-side UDP `iperf` intervals in the 10-60s time range, i.e., after the unfair station starts transmitting.

Unfair configuration	Fair BW (Mbit/s)	Unfair BW (Mbit/s)	Fair jitter (ms)	Unfair jitter (ms)	Fair loss (%)	Unfair loss (%)
Baseline/off	38.5	48.2	0.942	0.549	0	0
selfish_mode only	9.5	81.1	4.292	0.342	0.5193	0
disable_backoff only	4.1	83.7	11.135	0.370	13.42	0.00056
chanel_idle only	3.2	81.3	31.723	0.575	5.434	0.00116
selfish_txop_us=8100 only	21.0	65.7	2.034	0.432	0.0135	0
All options enabled	0.02	96.0	33.024	0.304	58.39	0

With both stations using the baseline configuration, the medium is shared imperfectly but still relatively evenly: after the unfair station starts, the fair station receives 38.5 Mbit/s and the other station receives

48.2 Mbit/s. Enabling each unfair mechanism separately shifts the channel share toward the unfair station. The EDCA manipulation alone reduces the fair station to 9.5 Mbit/s, while direct backoff bypass and forced channel-idle sensing are even more damaging, reducing the fair station to 4.1 Mbit/s and 3.2 Mbit/s, respectively. The TXOP-only case is less aggressive in this topology, because it mainly helps after the unfair station has already obtained the medium; nevertheless, the fair station still falls to 21.0 Mbit/s.

The strongest unfair behavior appears when all options are enabled together. In that run, over the 10-60s interval, the unfair station receives 96.0 Mbit/s, while the fair station drops to only 0.02 Mbit/s. The fair station also shows high jitter and packet loss, indicating unstable packet delivery. Figures 4–6 show the corresponding time-series plots for bandwidth, packet loss, and jitter.

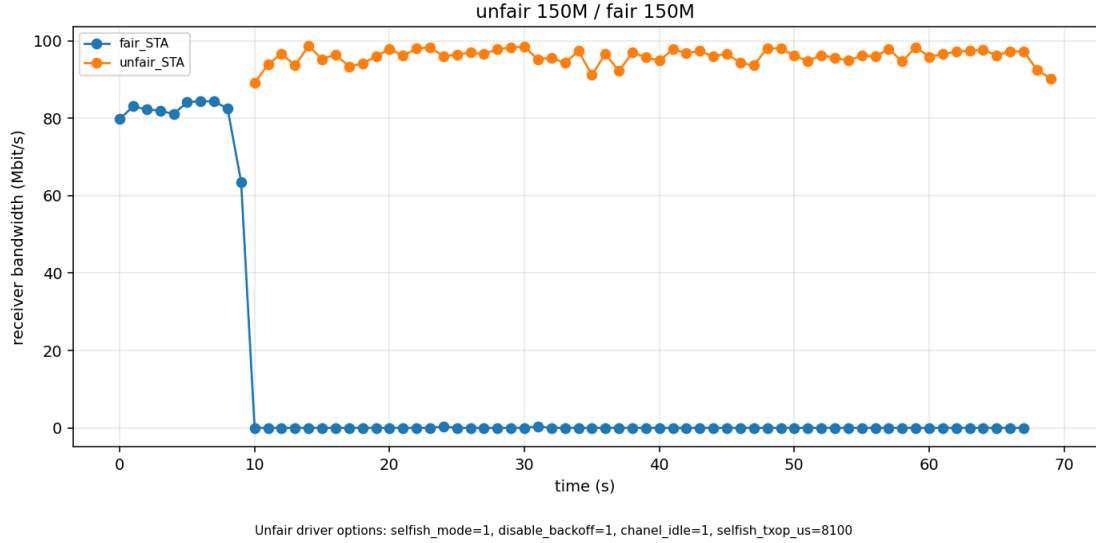


Figure 4: Experiment 3, all unfair options enabled: receiver bandwidth over time.

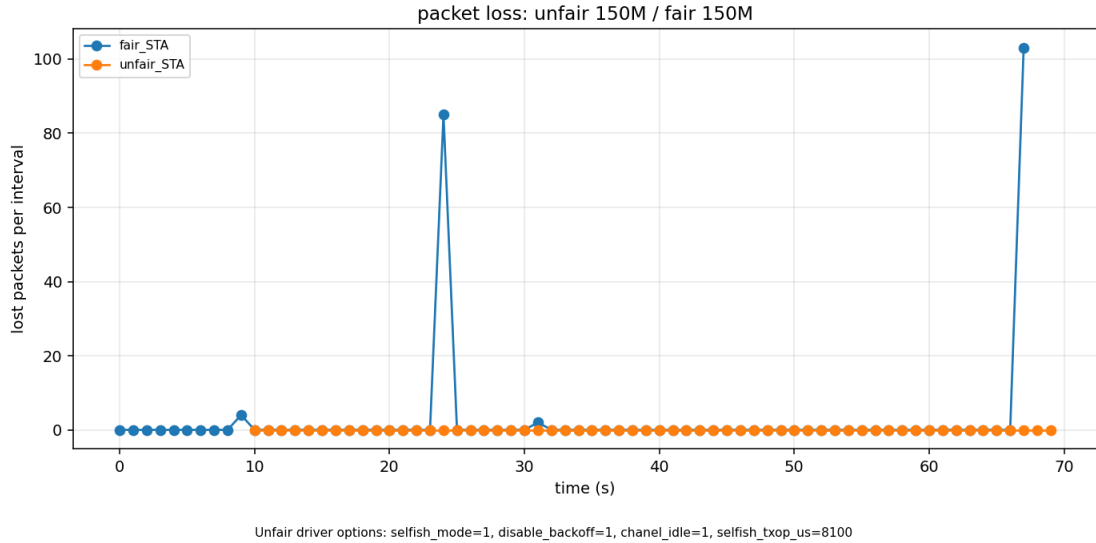


Figure 5: Experiment 3, all unfair options enabled: packet loss over time.

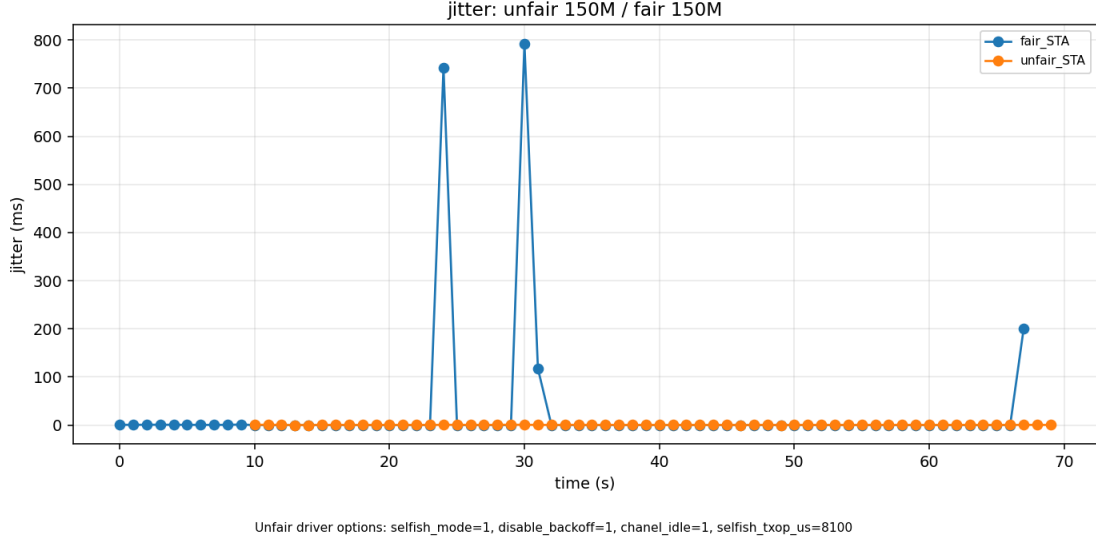


Figure 6: Experiment 3, all unfair options enabled: jitter over time.

5.3 Experiment 4: Two APs with Fair and Unfair Stations

In the fourth experiment, the fair station was connected to AP1 and the unfair station was connected to AP2. The two links used different SSIDs and IP subnets, but they still shared the same RF channel. This setup evaluates whether the unfair behavior also affects a separate BSS that is operating nearby on the same channel.

Table 3: Experiment 4 results: two APs, two stations on the same RF channel. Values are computed from receiver-side UDP `iperf` intervals in the 10-60s time range, i.e., after the unfair station starts transmitting.

Unfair configuration	Fair BW (Mbit/s)	Unfair BW (Mbit/s)	Fair jitter (ms)	Unfair jitter (ms)	Fair loss (%)	Unfair loss (%)
<code>selfish_mode</code> only	9.1	80.6	5.410	0.415	0.7846	0
<code>disable_backoff</code> only	4.9	82.0	9.022	0.393	6.671	0.00086
<code>channel_idle</code> only	3.5	78.9	25.413	0.562	4.585	0.00030
<code>selfish_txop_us=8100</code> only	20.8	64.0	2.036	0.526	0.00565	0.00187
All options enabled	0.12	92.9	120.852	0.379	34.56	0.00127

The same trend is visible even though the stations are now in different BSSs. The unfair link receives most of the channel capacity, while the fair link is strongly degraded. The TXOP-only case is again the least harmful of the tested unfair options, leaving the fair station at 20.8 Mbit/s. In contrast, backoff bypass and forced channel-idle sensing reduce the fair link to roughly 3.5-4.9 Mbit/s. This indicates that the unfair station does not need to be associated with the same AP to harm the fair station; sharing the same RF channel is enough for the modified MAC behavior to affect coexistence.

The all-options run is again the most aggressive case. Over the 10-60s interval, the unfair station receives 92.9 Mbit/s, while the fair station receives only 0.12 Mbit/s. The fair station's jitter remains much higher than the unfair station's jitter, which is consistent with long waiting periods and irregular packet delivery. Figures 7-9 show this run.

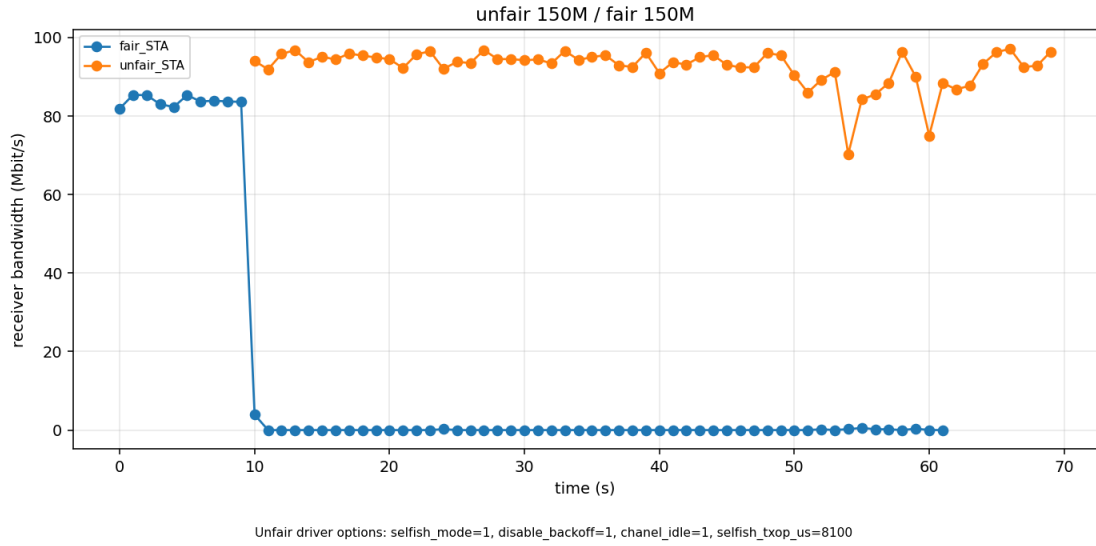


Figure 7: Experiment 4, all unfair options enabled: receiver bandwidth over time.

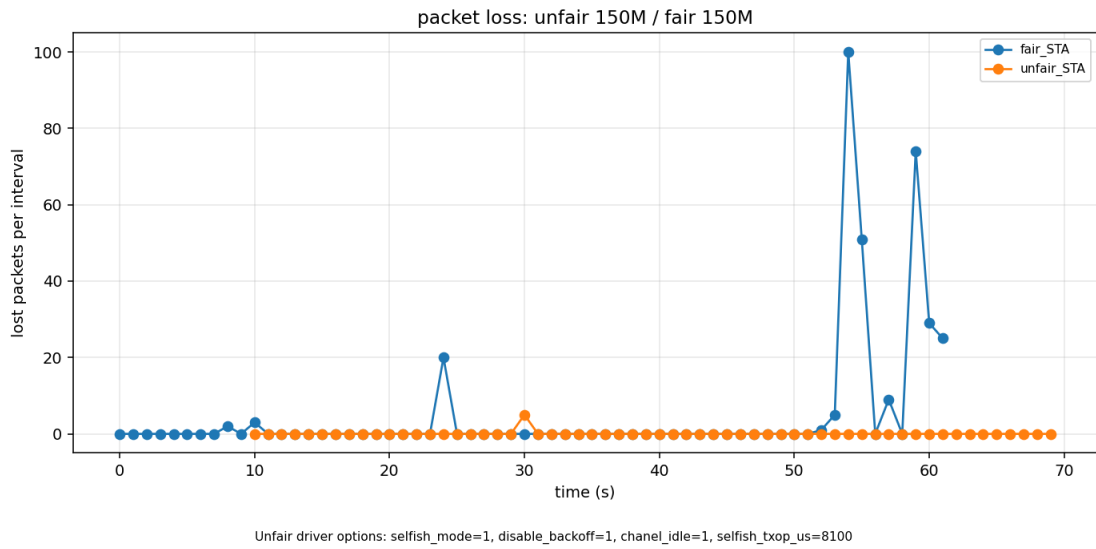


Figure 8: Experiment 4, all unfair options enabled: packet loss over time.

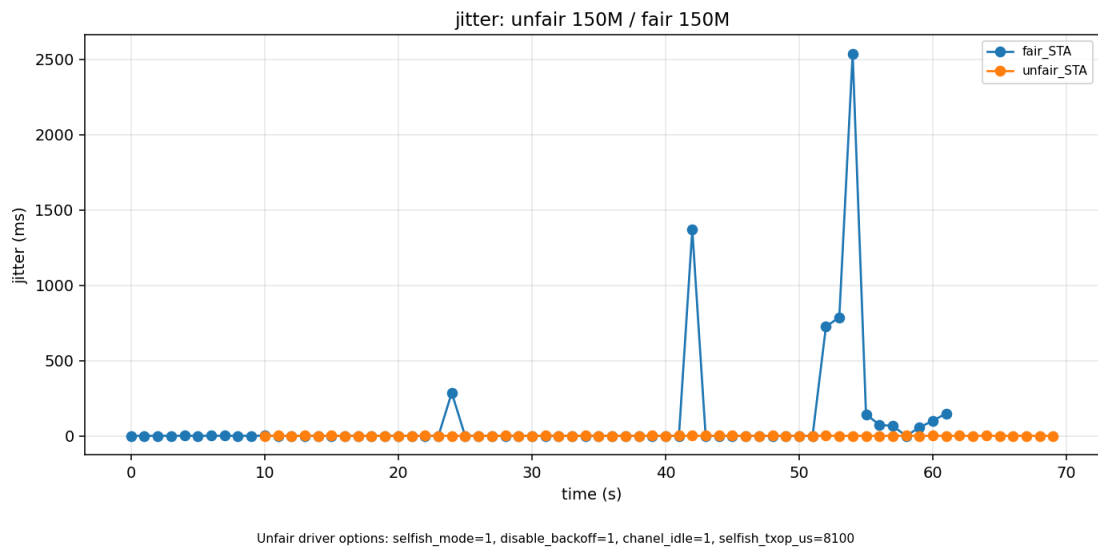


Figure 9: Experiment 4, all unfair options enabled: jitter over time.

5.4 Experiment 5: Channel-switch Follow with Backoff Bypass

The fifth experiment uses the two-AP topology with the channel-follow mechanism described in Section 3.7. AP1 and the fair station start on channel 1, and AP1 switches to channel 6 after 30 s. The unfair side runs a monitor/follow process that detects the AP1 channel-switch announcement and asks AP2 to move to the same target channel. In this run, the unfair station uses only the direct backoff-bypass option:

```
selfish_mode=0 disable_backoff=1 channel_idle=0 selfish_txop_us=0
```

After experimenting with the most aggressive configuration it was observed that the unfair station was jamming the CSA beacons of AP2, which is responsible for notifying the unfair node to change channel in order to follow the fair station. This resulted in the unfair station staying on the old channel 1 while the fair station had moved to channel 6. In some cases, the unfair node also jammed the CSA beacons of AP1, preventing the fair node from changing channels. For that reason, only the backoff-bypass option was enabled, as it allows the fair and unfair node to switch channels while also limiting the bandwidth of the fair node.

The switch-event logs show the AP1 switch command, the AP2 follow switch, and the hunter notification at approximately 30 s, followed by AP1 completing the switch to channel 6. Table 4 summarizes the receiver-side metrics over the 10-60s interval.

Table 4: Experiment 5 results: two APs with AP2 following AP1’s channel switch. Values are computed from receiver-side UDP iperf intervals in the 10-60s time range.

Flow	Avg. BW (Mbit/s)	Avg. jitter (ms)	Packet loss (%)	Lost/total datagrams
Fair station via AP1	4.16	14.662	17.743	3818/21518
Unfair station via AP2	94.97	0.379	0.00076	3/395766

The result shows that channel switching alone does not protect the fair link when the unfair side is able to follow the fair AP. After the unfair AP follows the move to channel 6, the two links continue sharing the same RF medium, and the unfair station with `disable_backoff=1` keeps most of the channel capacity. The fair flow averages only 4.16 Mbit/s during the 10-60s interval and has much higher packet loss and jitter than the unfair flow. Figures 10–12 show the corresponding time series.

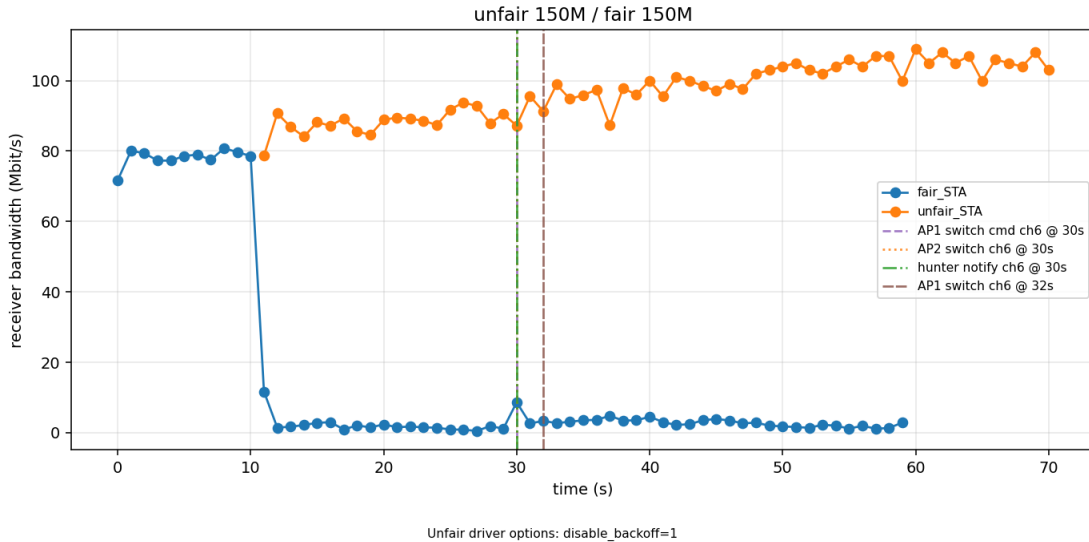


Figure 10: Experiment 5: receiver bandwidth with AP2 following AP1’s channel switch.

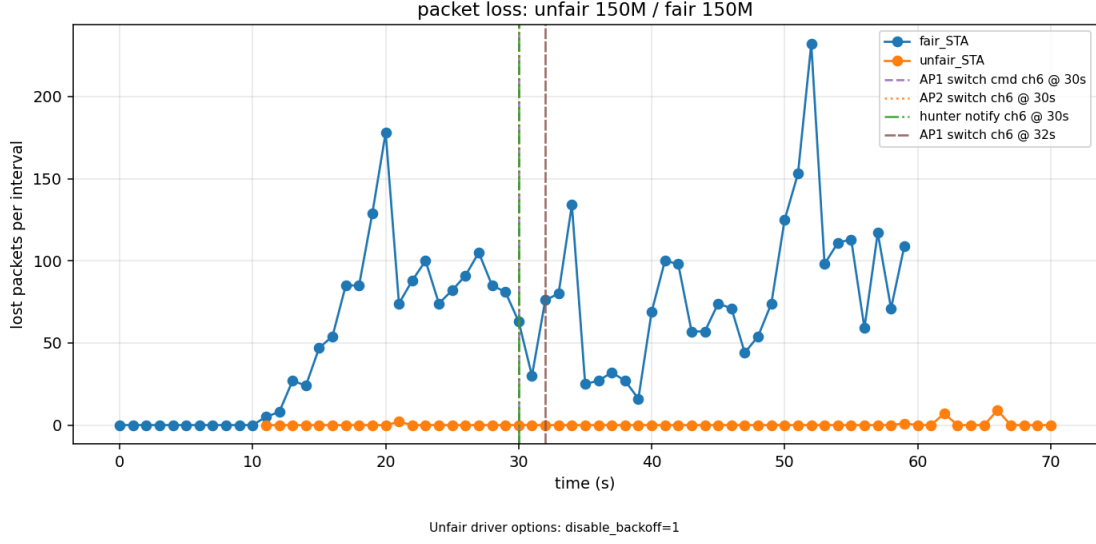


Figure 11: Experiment 5: packet loss with AP2 following AP1's channel switch.

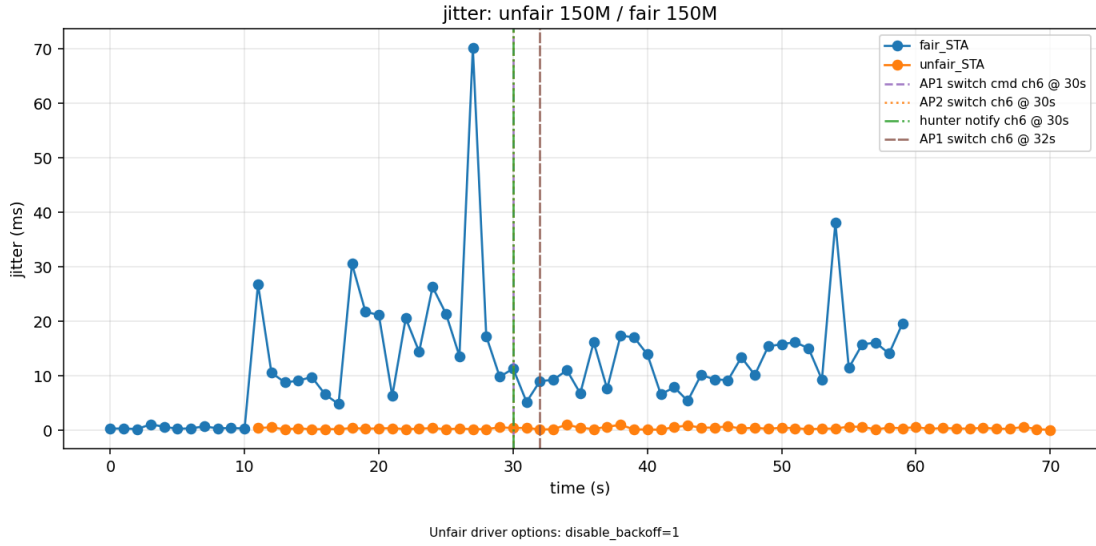


Figure 12: Experiment 5: jitter with AP2 following AP1's channel switch.

5.5 Packet-loss Interpretation

A notable observation is that the packet-loss plots mainly show losses for the fair station, while the unfair station often has zero or almost zero loss. A plausible explanation is that the fair station repeatedly loses access opportunities because it continues to obey backoff and carrier-sensing rules. While it waits, the UDP `iperf` client can keep producing packets faster than the fair link can transmit them. This can fill the operating-system or driver transmit buffers, so older datagrams may be dropped before they ever reach the AP. In that case, the AP-side `iperf` server reports missing UDP datagrams for the fair flow, whereas the unfair flow does not show the same loss because it obtains channel access more frequently.

This is a reasonable assumption, but it should be treated as an interpretation rather than a proven mechanism. The receiver-side `iperf` logs show that datagrams are missing, but they do not distinguish whether the missing packets were lost due to wireless collisions, MAC retries being exhausted, driver queue drops, or socket-buffer overflow at the sender. Proving the exact cause would require extra evidence such as monitor-mode captures, retry counters, queue statistics, or driver debug counters. Therefore, in this report we use the

buffer/backoff explanation as a likely cause that is consistent with the observed behavior, while avoiding a stronger claim than the available logs support.

6 Conclusion

This project demonstrated how modifying the ath9k driver can change the behavior of a station at the MAC layer and give it an unfair advantage in an IEEE 802.11 network. By changing contention parameters, backoff behavior, channel sensing, and TXOP duration, the modified station was able to access the shared medium more aggressively than fair stations. The experiments showed that these low-level changes can significantly affect throughput distribution and wireless coexistence.

As future work, the experiments could be extended with more repetitions, additional traffic patterns, and more detailed wireless measurements such as packet captures, retry counters, and driver statistics. This would make it possible to analyze the exact causes of performance changes more precisely and could also support future work on detecting or mitigating selfish MAC behavior.

References

- [1] M. Vanhoef and F. Piessens, “Advanced wi-fi attacks using commodity hardware,” in *Proceedings of the 30th Annual Computer Security Applications Conference*, ser. ACSAC '14. ACM, 2014, pp. 256–265.
- [2] NITlab, “NITOS - NITlab - Network Implementation Testbed Laboratory,” <https://nitlab.inf.uth.gr/NITlab/nitos>.